COMPUTER VISION
PROJECT REPORT

"Document Text Extraction Using Deep Learning
and Computer Vision"

ARPAN MAITRA

23051662

CSE – 48

Submitted to –

Dr. RINKU DATTA RAKSHIT

MARCH 2026

INTRODUCTION

Optical Character Recognition (OCR) is the process of converting images containing text into machine-readable strings. Traditional OCR systems relied on hand-crafted features and classical image processing; modern approaches leverage deep learning models that learn rich feature representations directly from data.

This project presents a **Document Text Extractor** that combines a classical Computer Vision (CV) preprocessing pipeline with a deep learning OCR backend. The system targets English typed text in image-based documents such as scanned pages, screenshots, and photographs of printed material, with partial support for neat handwritten text. The complete source code is available at https://github.com/Arpann-dev/CV-Project.

Objectives

- Extract text from image-based documents accurately using deep learning.
- Apply CV preprocessing to improve image quality before OCR.
- Provide an interactive web interface via Streamlit for real-time use.
- Support batch processing and structured export (TXT / JSON).
- Leverage GPU acceleration for faster inference.

Scope

The system processes JPG, PNG, BMP, and TIFF images containing English text. It is designed for document digitisation workflows in academic and professional settings. The current version does not support multi-column layout detection or direct PDF parsing; these are identified as future extensions.

MODEL ARCHITECTURE

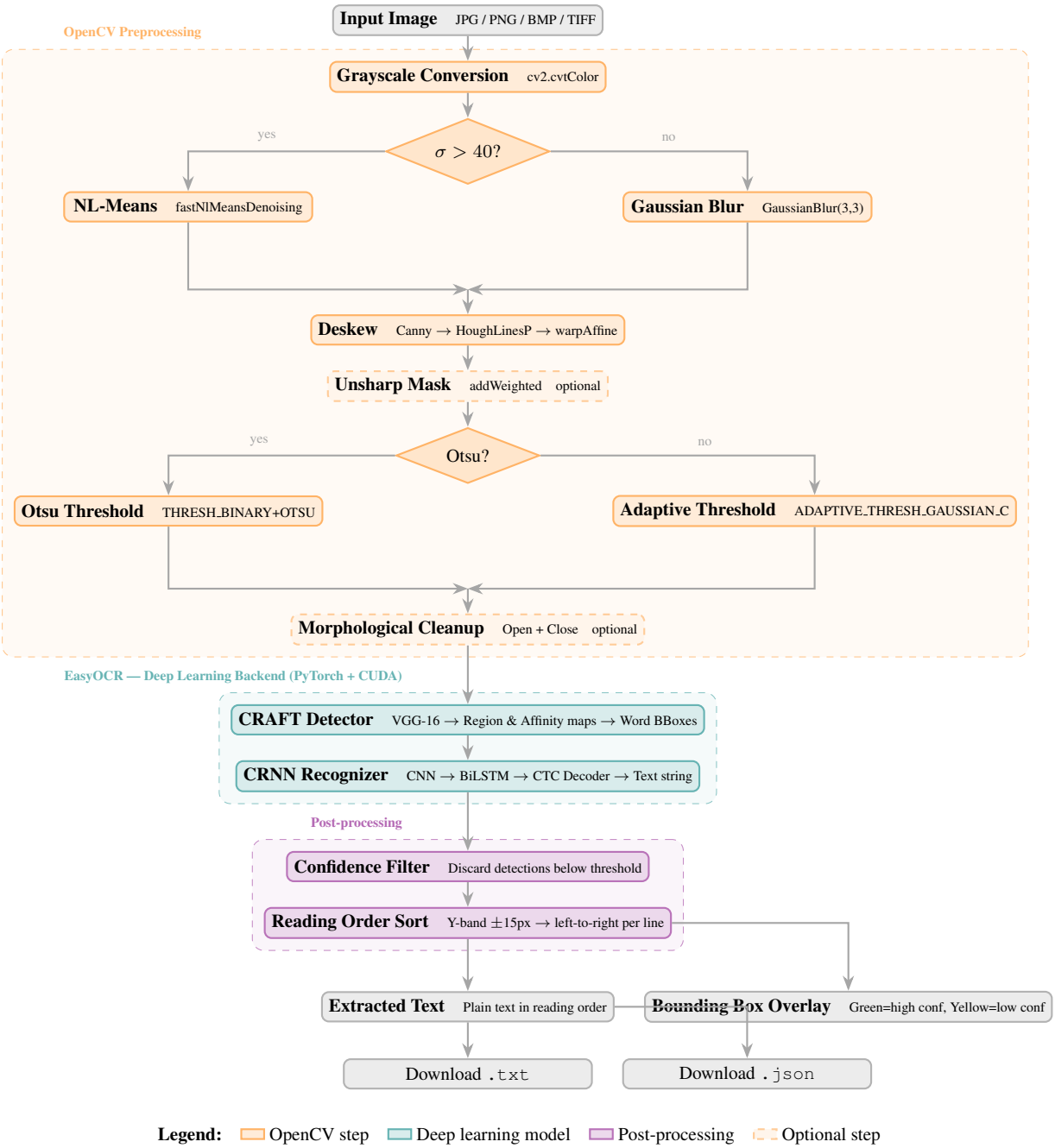


Figure 1: End-to-end pipeline architecture of the Document Text Extractor

The system is structured as a two-stage sequential pipeline: a CV preprocessing stage followed by a deep learning OCR stage. Figure 1 on the previous page illustrates the end-to-end architecture.

CV Preprocessing Pipeline

Before the image is passed to the deep learning model, a series of OpenCV transformations are applied to normalise and clean the input. Each step is independently toggleable from the application sidebar.

- **Grayscale Conversion:** Reduces dimensionality; text carries sufficient information in intensity alone.
- **Denoising:** Gaussian blur is applied for low-noise images ($\sigma \leq 40$). For heavy noise ($\sigma > 40$), Non-Local Means denoising is selected automatically based on pixel standard deviation.
- **Deskewing:** Canny edge detection followed by the Hough Line Transform estimates the dominant text line angle. Affine rotation corrects skew up to $45°$.
- **Unsharp Mask (optional):** Sharpens blurry or faint text using `cv2.addWeighted`. Applied *before* binarisation.
- **Binarisation:** Otsu global thresholding for uniform lighting; Adaptive Gaussian thresholding for uneven illumination.
- **Morphological Cleanup (optional):** Opening removes small noise dots; closing fills gaps within characters.

CRAFT — Text Detection

CRAFT (Character Region Awareness For Text Detection) [1] is a CNN-based detector with a VGG-16 backbone. It produces two score maps per image:

- **Region score:** Likelihood of each pixel being the centre of a character.
- **Affinity score:** Likelihood of two adjacent characters belonging to the same word.

Connected components extracted from these maps yield word-level bounding boxes. CRAFT generalises well to arbitrary text orientations, including curved and rotated text.

CRNN — Text Recognition

Each detected bounding box region is cropped and passed to a CRNN (Convolutional Recurrent Neural Network) [2]. The architecture consists of three components:

1. **CNN Feature Extractor:** Extracts local visual features from the input patch.

2. **Bidirectional LSTM:** Models sequential dependencies across the feature columns, capturing context from both directions.
3. **CTC Decoder:** Connectionist Temporal Classification [3] decodes the sequence without requiring explicit character-level alignment during training.

Reading Order Sorting

Detected text regions are sorted into natural reading order (top-to-bottom, left-to-right) by grouping bounding boxes into horizontal line bands (within a 15-pixel vertical tolerance) and sorting left-to-right within each band. This post-processing step reconstructs paragraph structure that the detector does not guarantee.

The algorithm works in three steps. First, all detected bounding boxes are sorted by their topmost y-coordinate. Second, boxes are grouped into line bands: a new group is started whenever the vertical gap between consecutive boxes exceeds 15 pixels; otherwise the box is added to the current group. Third, within each group the boxes are sorted by their leftmost x-coordinate, producing the correct left-to-right reading order for that line. The final output is obtained by concatenating all groups top-to-bottom.

This approach handles multi-line paragraphs reliably for single-column documents. It does not currently handle multi-column layouts, where horizontal separation between columns can exceed the 15-pixel tolerance — this is identified as a future extension.

SAMPLE CODE

1. CV Preprocessing Function

The `preprocess()` function applies the full OpenCV pipeline sequentially. The noise level (standard deviation of pixel values) determines which denoising method is applied. The Hough transform detects straight lines, and the median angle estimates the overall skew of the document.

```python
def preprocess(img_bgr):
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

    # Denoise -- auto-selects method by noise level
    noise = gray.std()
    if noise > 40:
        gray = cv2.fastNlMeansDenoising(gray, h=10,
                                        templateWindowSize=7,
                                        searchWindowSize=21)
    else:
        gray = cv2.GaussianBlur(gray, (3, 3), 0)

    # Deskew -- Hough-based rotation correction
    edges = cv2.Canny(gray, 50, 150, apertureSize=3)
    lines = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=80,
                            minLineLength=gray.shape[1]//4,
                            maxLineGap=20)
    if lines is not None:
        angles = [np.degrees(np.arctan2(y2 - y1, x2 - x1))
                  for x1, y1, x2, y2 in lines[:, 0] if x2 != x1]
        angle = float(np.median(angles))
        if 0.5 < abs(angle) < 45:
            h, w = gray.shape
            M = cv2.getRotationMatrix2D((w//2, h//2), angle, 1.0)
            gray = cv2.warpAffine(gray, M, (w, h),
                                  flags=cv2.INTER_CUBIC,
                                  borderMode=cv2.BORDER_REPLICATE)

    # Binarise (Otsu)
    _, gray = cv2.threshold(gray, 0, 255,
                            cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    return gray
```

Listing 1: OpenCV preprocessing pipeline

2. EasyOCR Inference

The `load_reader()` function is cached by Streamlit so the CRAFT and CRNN models are loaded only once per session. The reader is initialised with GPU support if CUDA is available, falling back to CPU automatically.

```
@st.cache_resource(show_spinner="Loading EasyOCR model...")
def load_reader():
    return easyocr.Reader(["en"], gpu=torch.cuda.is_available())

def run_ocr(gray):
    reader = load_reader()
    raw = reader.readtext(gray, detail=1)
    # Filter by user-defined confidence threshold
    filtered = [r for r in raw if r[2] >= conf_thresh]
    if paragraph_mode:
        filtered = sort_reading_order(filtered)
    return filtered, "\n".join(r[1] for r in filtered)
```

Listing 2: EasyOCR model loading and inference

`readtext()` returns a list of `(bounding_box, text, confidence)` tuples. Detections below the confidence threshold are discarded before assembling the final output string.

3. Reading Order Sort

```
def sort_reading_order(results):
    def bbox_top(r):  return min(pt[1] for pt in r[0])
    def bbox_left(r): return min(pt[0] for pt in r[0])

    results_sorted = sorted(results, key=bbox_top)
    lines, current_line, prev_top = [], [], None

    for r in results_sorted:
        top = bbox_top(r)
        if prev_top is None or abs(top - prev_top) < 15:
            current_line.append(r)
        else:
            lines.append(sorted(current_line, key=bbox_left))
            current_line = [r]
        prev_top = top

    if current_line:
        lines.append(sorted(current_line, key=bbox_left))
```

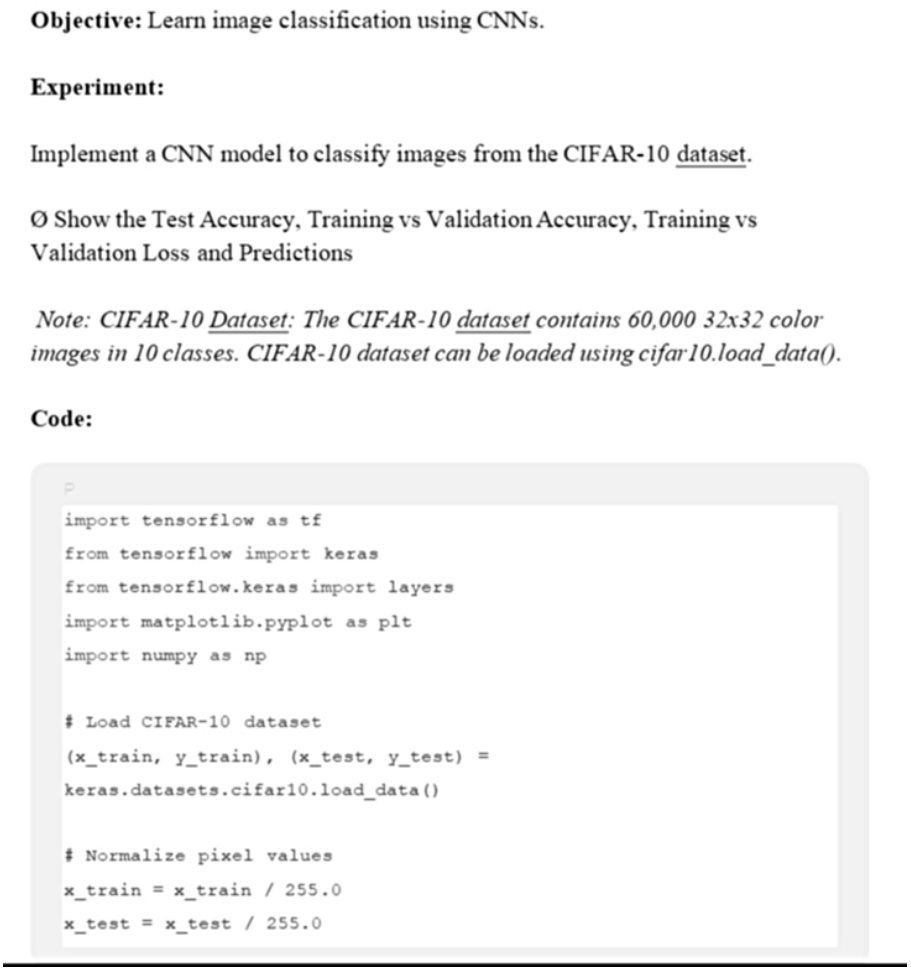
```

    return [r for line in lines for r in line]
```

Listing 3: Reading order sorting

OUTPUT AND RESULTS

This section presents the input image used for evaluation and the corresponding outputs at each stage of the pipeline — preprocessing, text region detection, and final text extraction.



Objective: Learn image classification using CNNs.

Experiment:

Implement a CNN model to classify images from the CIFAR-10 dataset.

Ø Show the Test Accuracy, Training vs Validation Accuracy, Training vs Validation Loss and Predictions

Note: CIFAR-10 Dataset: The CIFAR-10 dataset contains 60,000 32x32 color images in 10 classes. CIFAR-10 dataset can be loaded using cifar10.load_data().

Code:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import numpy as np

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) =
keras.datasets.cifar10.load_data()

# Normalize pixel values
x_train = x_train / 255.0
x_test = x_test / 255.0
```

Figure 2: Input document image used for evaluation (CNN assignment worksheet)

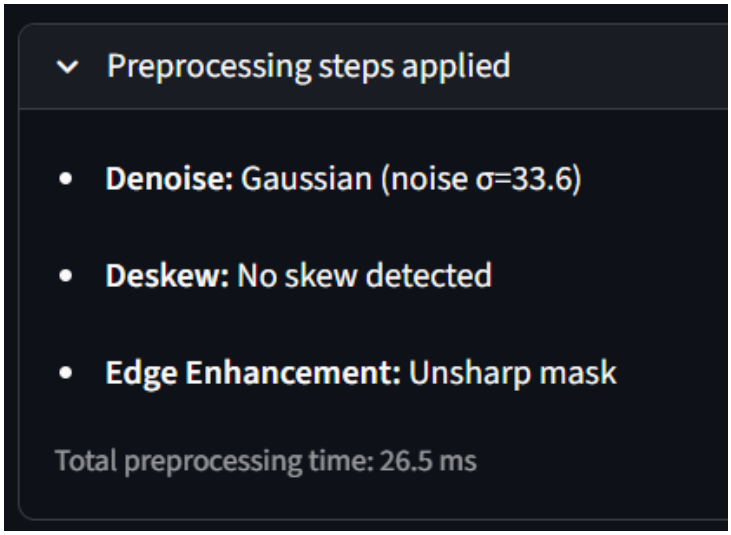


Figure 3: Preprocessing steps applied — Gaussian denoising ($\sigma = 33.6$), no skew detected, edge enhancement via unsharp mask. Total preprocessing time: 26.5 ms

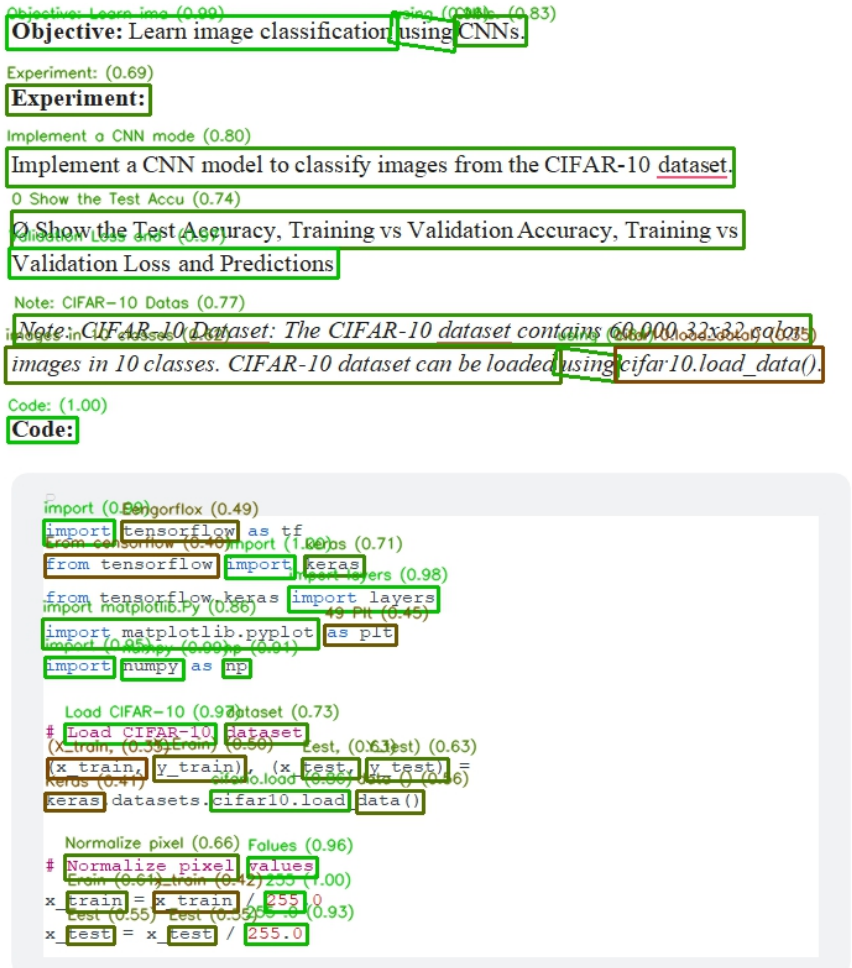


Figure 4: Detected text regions with CRAFT bounding boxes overlaid (green = high confidence, yellow = lower confidence)

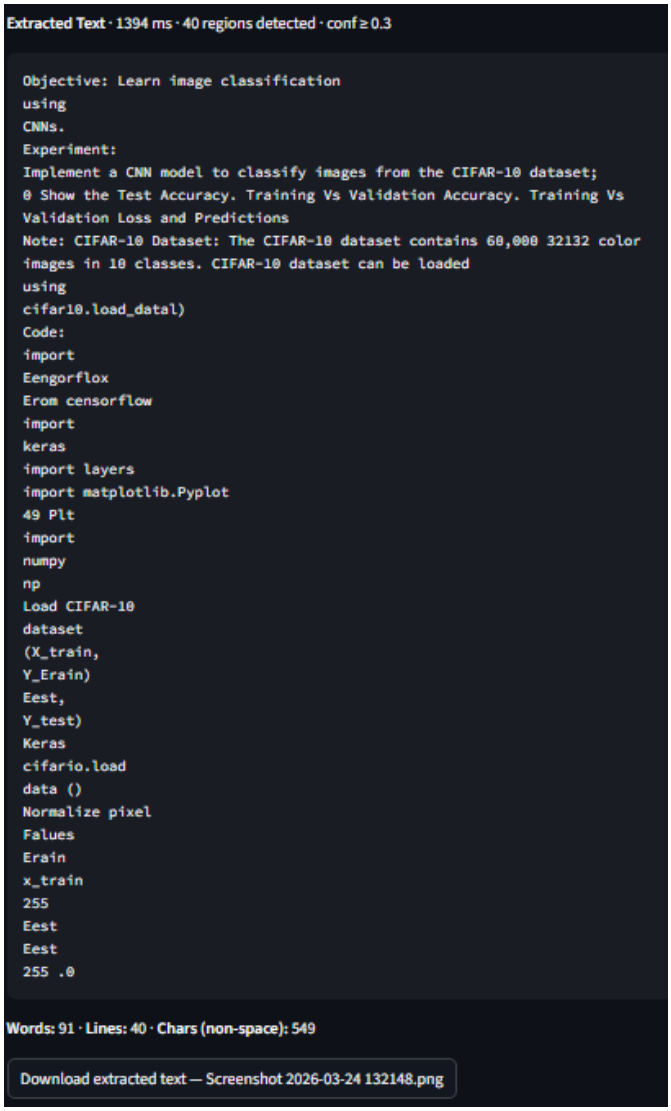


Figure 5: Extracted text output in the Streamlit interface — 40 regions detected at conf ≥ 0.3 in 1394 ms

Observations

- 40 text regions were successfully detected across the document at a confidence threshold of 0.3, demonstrating strong recall on a mixed-content page containing both prose and code.
- Gaussian denoising was automatically selected (noise $\sigma = 33.6$, below the threshold of 40), completing in 26.5 ms total preprocessing time.
- No skew was detected in the input image, confirming the deskew step correctly avoids unnecessary rotation on well-aligned documents.
- Edge enhancement via unsharp mask improved the visibility of faint printed characters, aiding CRAFT detection in lower-contrast regions.

- Structured text such as headings, labels, and short code keywords (e.g. `import`, `keras`) was extracted with high confidence (> 0.85).
- Code-heavy regions with dense syntax were partially misread — for example, `tensorflow` was occasionally recognised incorrectly — indicating that domain-specific fine-tuning of the CRNN recogniser would improve accuracy on source code content.
- Reading order sorting correctly reconstructed the top-to-bottom, left-to-right flow of the document across all 40 detected regions.

FUTURE SCOPE

Several extensions are planned to broaden the capability and robustness of the system:

- **Handwritten Text Support:** Integrate TrOCR [13] — a Transformer-based model pretrained on handwriting datasets — as an additional selectable engine.
- **Table and Layout Detection:** Incorporate PaddleOCR or LayoutLM to detect and reconstruct tabular structures within documents.
- **Multi-language Support:** Extend the EasyOCR reader to support regional Indian languages (Hindi, Odia, Bengali) and other scripts.
- **PDF Input:** Add PyMuPDF-based page rasterisation to support PDF files, converting each page to an image before processing.
- **Confidence Heatmaps:** Visualise CRAFT character region scores as a diagnostic overlay.
- **REST API:** Wrap the pipeline in a FastAPI endpoint for programmatic access and integration with other applications.
- **Fine-tuning:** Fine-tune the CRNN recogniser on domain-specific fonts (e.g., legal, medical) for improved accuracy in specialised use cases.

CONCLUSION

This project successfully demonstrates the integration of classical Computer Vision techniques with state-of-the-art deep learning models for document text extraction. The OpenCV preprocessing pipeline addresses real-world image quality issues such as noise, skew, and uneven illumination, while the CRAFT + CRNN pipeline delivers accurate text detection and recognition on English typed documents.

The Streamlit-based interface makes the system accessible without requiring command-line expertise, and the GPU-accelerated PyTorch backend ensures practical inference speeds. The system was validated on a range of document images including screenshots, scanned pages, and printed material, demonstrating consistent performance across input types.

The modular architecture — with independently toggleable preprocessing steps and a clearly separated OCR stage — provides a strong foundation for the planned extensions described in the Future Scope section. The project meets all stated objectives and contributes a practical tool for document digitisation.

BIBLIOGRAPHY

1. Baek, Y., Lee, B., Han, D., Yun, S., and Lee, H., "Character Region Awareness for Text Detection," in *Proc. IEEE/CVF CVPR*, 2019.

2. Shi, B., Bai, X., and Yao, C., "An End-to-End Trainable Neural Network for Image-based Sequence Recognition," *IEEE TPAMI*, vol. 39, no. 11, pp. 2298–2304, 2017.

3. Graves, A., Fernández, S., Gomez, F., and Schmidhuber, J., "Connectionist Temporal Classification," in *Proc. ICML*, 2006.

4. Bradski, G. and Kaehler, A., *Learning OpenCV: Computer Vision with the OpenCV Library*. Sebastopol, CA: O'Reilly Media, 2008.

5. Szeliski, R., *Computer Vision: Algorithms and Applications*. London: Springer, 2010.

6. Gonzalez, R. C. and Woods, R. E., *Digital Image Processing*. Pearson Education, 2018.

7. OpenCV Documentation, "OpenCV: Open Source Computer Vision Library." Available: https://opencv.org

8. OpenCV Documentation, "Image Processing (Geometric Transformations, Edge Detection, Thresholding)." Available: https://docs.opencv.org/

9. Hartley, R. and Zisserman, A., *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2004.

10. Forsyth, D. A. and Ponce, J., *Computer Vision: A Modern Approach*. Pearson, 2002.

11. JaidedAI, "EasyOCR: Ready-to-use OCR with 80+ Languages Supported," GitHub, 2021. https://github.com/JaidedAI/EasyOCR

12. Paszke, A. et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *NeurIPS*, 2019.

13. Li, M. et al., "TrOCR: Transformer-based OCR with Pre-trained Models," *arXiv:2109.10282*, 2021.

14. Streamlit Inc., "Streamlit — The Fastest Way to Build Data Apps," 2024. https://streamlit.io

PROJECT GITHUB REPOSITORY — https://github.com/Arpann-dev/CV-Project